



Divide & Conquer

Lecture 2 | Algorithms Design & Analysis

Girish Varma
IIIT Hyderabad

Algo Design Idea I: Recursion



Algo Design Idea II: Divide & Conquer



Multiplication Again



$$x = \boxed{x_L} \boxed{x_R} = 2^{n/2}x_L + x_R$$

$$y = \boxed{y_L} \boxed{y_R} = 2^{n/2}y_L + y_R.$$

For instance, if $x = 10110110_2$ (the subscript 2 means “binary”) then $x_L = 1011_2$, $x_R = 0110_2$, and $x = 1011_2 \times 2^4 + 0110_2$. The product of x and y can then be rewritten as

$$xy = (2^{n/2}x_L + x_R)(2^{n/2}y_L + y_R) = 2^n x_L y_L + 2^{n/2} (x_L y_R + x_R y_L) + x_R y_R.$$

$$T(n) = 4T(n/2) + O(n).$$

Gauss's Complex Multiplication Trick



$$(a + bi)(c + di) = ac - bd + (bc + ad)i$$

-

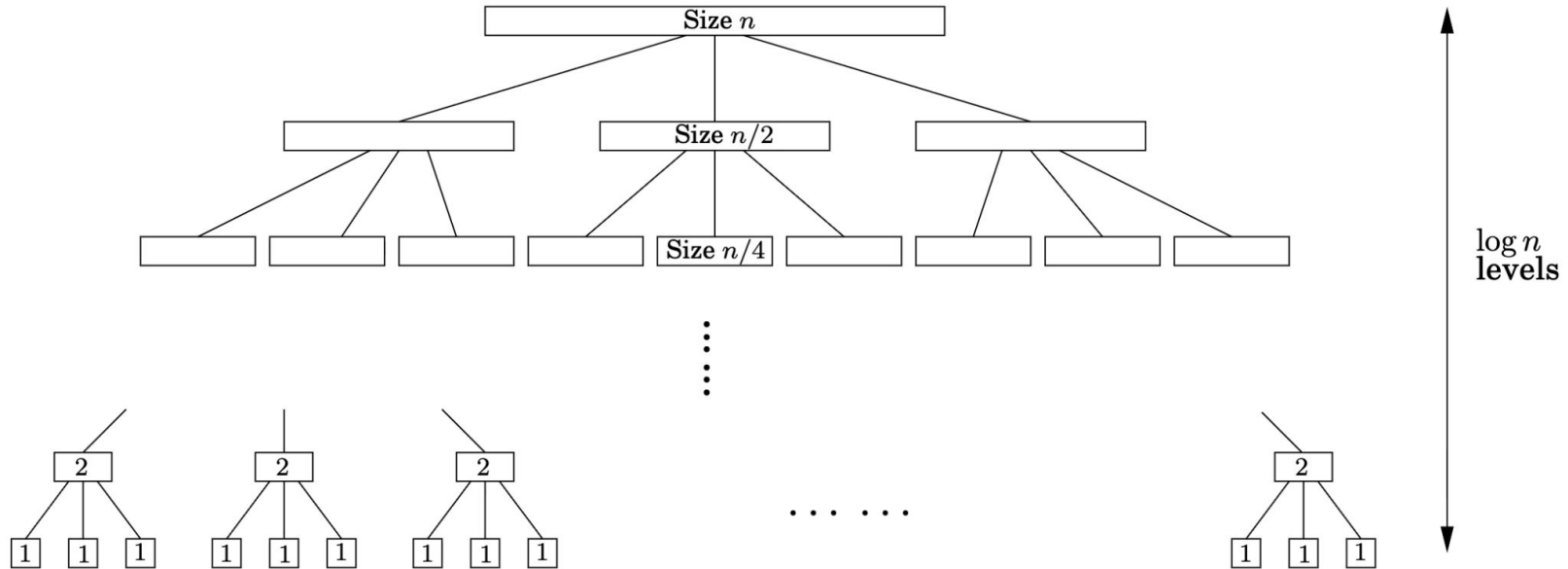
$$bc + ad = (a + b)(c + d) - ac - bd.$$


```

return  $P_1 \times 2^n + (P_3 - P_1 - P_2) \times 2^{n/2} + P_2$ 

```

$$T(n) = 3T(n/2) + O(n).$$



$$3^k \times O\left(\frac{n}{2^k}\right) = \left(\frac{3}{2}\right)^k \times O(n).$$

Master Theorem for Recursion



Master theorem² *If $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ for some constants $a > 0$, $b > 1$, and $d \geq 0$, then*

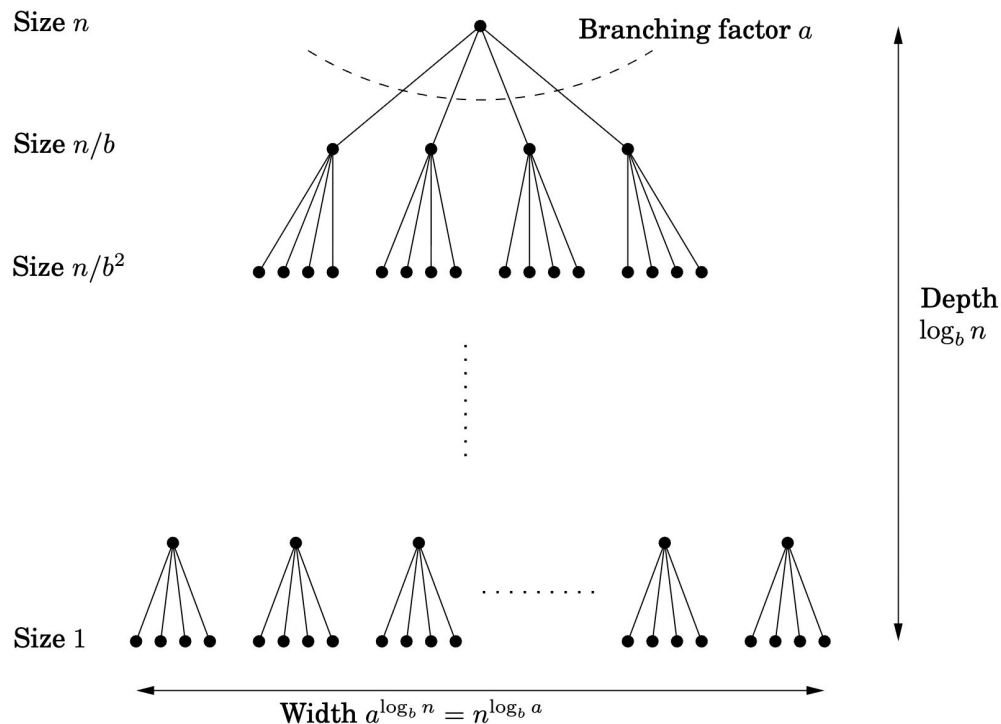
$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a . \end{cases}$$

Master Theorem for Recursion



Figure 2.3 Each problem of size n is divided into a subproblems of size n/b .

$$a^k \times O\left(\frac{n}{b^k}\right)^d = O(n^d) \times \left(\frac{a}{b^d}\right)^k.$$



Master Theorem for Recursion



As k goes from 0 (the root) to $\log_b n$ (the leaves), these numbers form a geometric series with ratio a/b^d . Finding the sum of such a series in big- O notation is easy (Exercise 0.2), and comes down to three cases.

1. *The ratio is less than 1.*

Then the series is decreasing, and its sum is just given by its first term, $O(n^d)$.

2. *The ratio is greater than 1.*

The series is increasing and its sum is given by its last term, $O(n^{\log_b a})$:

$$n^d \left(\frac{a}{b^d} \right)^{\log_b n} = n^d \left(\frac{a^{\log_b n}}{(b^{\log_b n})^d} \right) = a^{\log_b n} = a^{(\log_a n)(\log_b a)} = n^{\log_b a}.$$

3. *The ratio is exactly 1.*

In this case all $O(\log n)$ terms of the series are equal to $O(n^d)$.

MergeSort



```
function mergesort( $a[1 \dots n]$ )
```

Input: An array of numbers $a[1 \dots n]$

Output: A sorted version of this array

```
if  $n > 1$ :
```

```
    return merge(mergesort( $a[1 \dots \lfloor n/2 \rfloor]$ ), mergesort( $a[\lfloor n/2 \rfloor + 1 \dots n]$ ))
```

```
else:
```

```
    return  $a$ 
```

```
function merge( $x[1 \dots k], y[1 \dots l]$ )
```

```
if  $k = 0$ : return  $y[1 \dots l]$ 
```

```
if  $l = 0$ : return  $x[1 \dots k]$ 
```

```
if  $x[1] \leq y[1]$ :
```

```
    return  $x[1] \circ \text{merge}(x[2 \dots k], y[1 \dots l])$ 
```

```
else:
```

```
    return  $y[1] \circ \text{merge}(x[1 \dots k], y[2 \dots l])$ 
```

Running Time



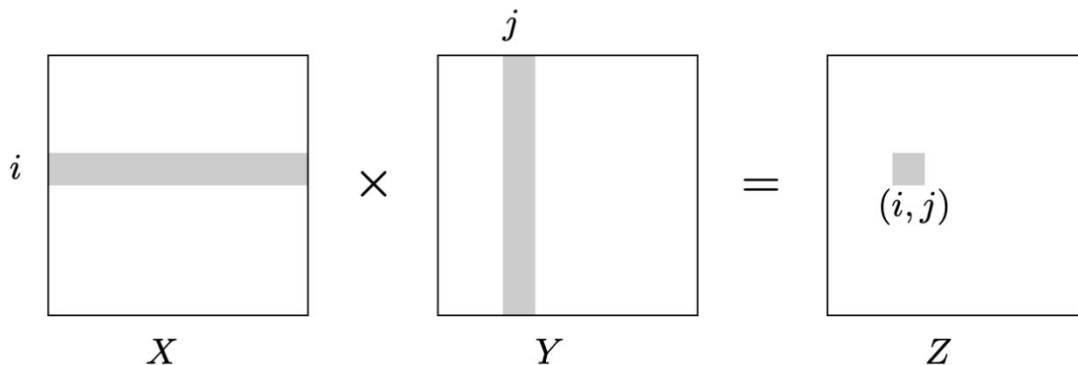
Matrix Multiplication



The product of two $n \times n$ matrices X and Y is a third $n \times n$ matrix $Z = XY$, with (i, j) th entry

$$Z_{ij} = \sum_{k=1}^n X_{ik} Y_{kj}.$$

To make it more visual, Z_{ij} is the dot product of the i th row of X with the j th column of Y :



Matrix Multiplication



Matrix multiplication is particularly easy to break into subproblems, because it can be performed *blockwise*. To see what this means, carve X into four $n/2 \times n/2$ blocks, and also Y :

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix}, \quad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}.$$

Then their product can be expressed in terms of these blocks and is exactly as if the blocks were single elements (Exercise 2.11).

$$XY = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}$$

Matrix Multiplication



It turns out XY can be computed from just *seven* $n/2 \times n/2$ subproblems, via a decomposition so tricky and intricate that one wonders how Strassen was ever able to discover it!

$$XY = \begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{bmatrix}$$

where

$$P_1 = A(F - H)$$

$$P_2 = (A + B)H$$

$$P_3 = (C + D)E$$

$$P_4 = D(G - E)$$

$$P_5 = (A + D)(E + H)$$

$$P_6 = (B - D)(G + H)$$

$$P_7 = (A - C)(E + F)$$

Read



Chapter 2 upto Section 2.5 in

<http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf>

Solve



Exercises 2.5, 2.19, 2.27, 2.28, 2.32 in

<http://algorithmics.isi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf>